Practice program

1)

```
#include <stdio.h>
int linearSearch(int arr[], int n, int key) {
    int i;
    for(i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;

    while(low <= high) {
        mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int n, i, key, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("\n1. Linear Search\n");
    printf("2. Binary Search (Array must be sorted)\n");
```

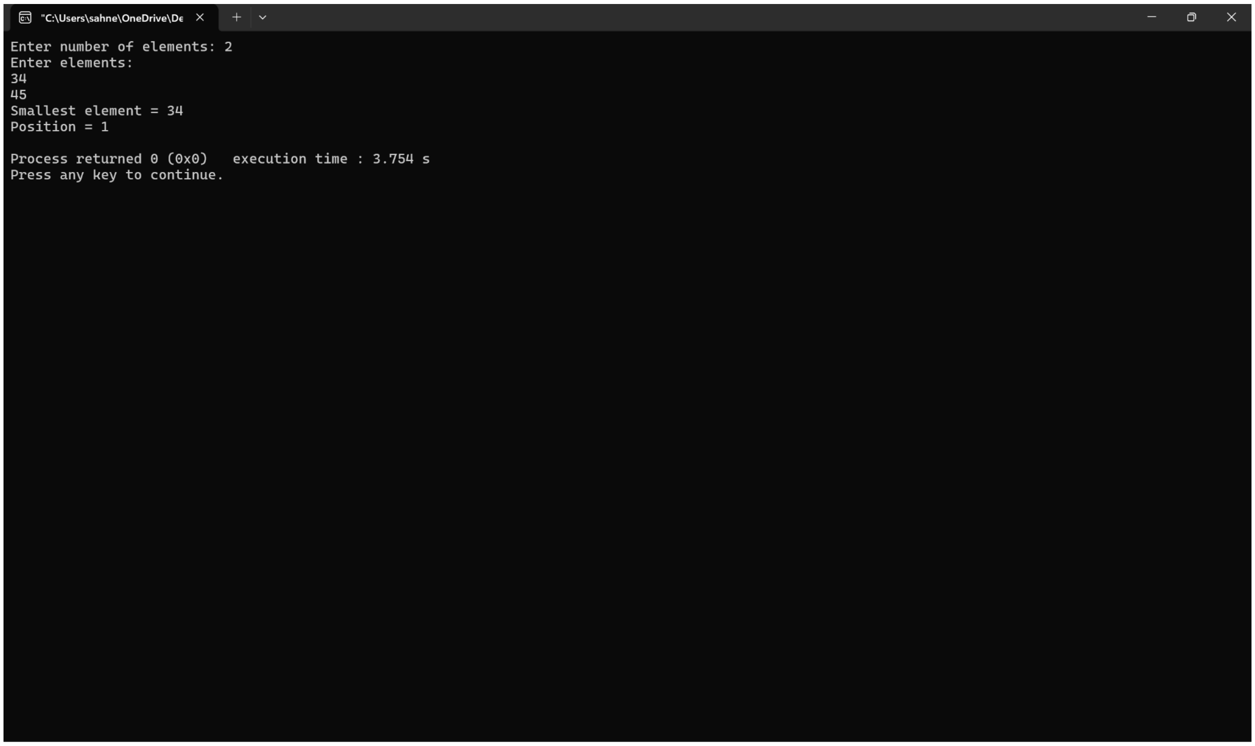
```c
    printf("Enter your choice: ");
    scanf("%d", &choice);

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result;

    switch(choice) {
        case 1:
            result = linearSearch(arr, n, key);
            if(result != -1)
                printf("Element found at position %d\n", result + 1);
            else
                printf("Element not found\n");
            break;

        case 2:
            result = binarySearch(arr, n, key);
            if(result != -1)
                printf("Element found at position %d\n", result + 1);
            else
                printf("Element not found\n");
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

```
"C:\Users\sahne\OneDrive\De

Enter number of elements: 2
Enter elements:
12
23

1. Linear Search
2. Binary Search (Array must be sorted)
Enter your choice: 2
Enter element to search: 23
Element found at position 2

Process returned 0 (0x0)   execution time : 13.518 s
Press any key to continue.
```

```
"C:\Users\sahne\OneDrive\De

Enter number of elements: 2
Enter elements:
12
23

1. Linear Search
2. Binary Search (Array must be sorted)
Enter your choice: 1
Enter element to search: 12
Element found at position 1

Process returned 0 (0x0)   execution time : 11.228 s
Press any key to continue.
```

2)

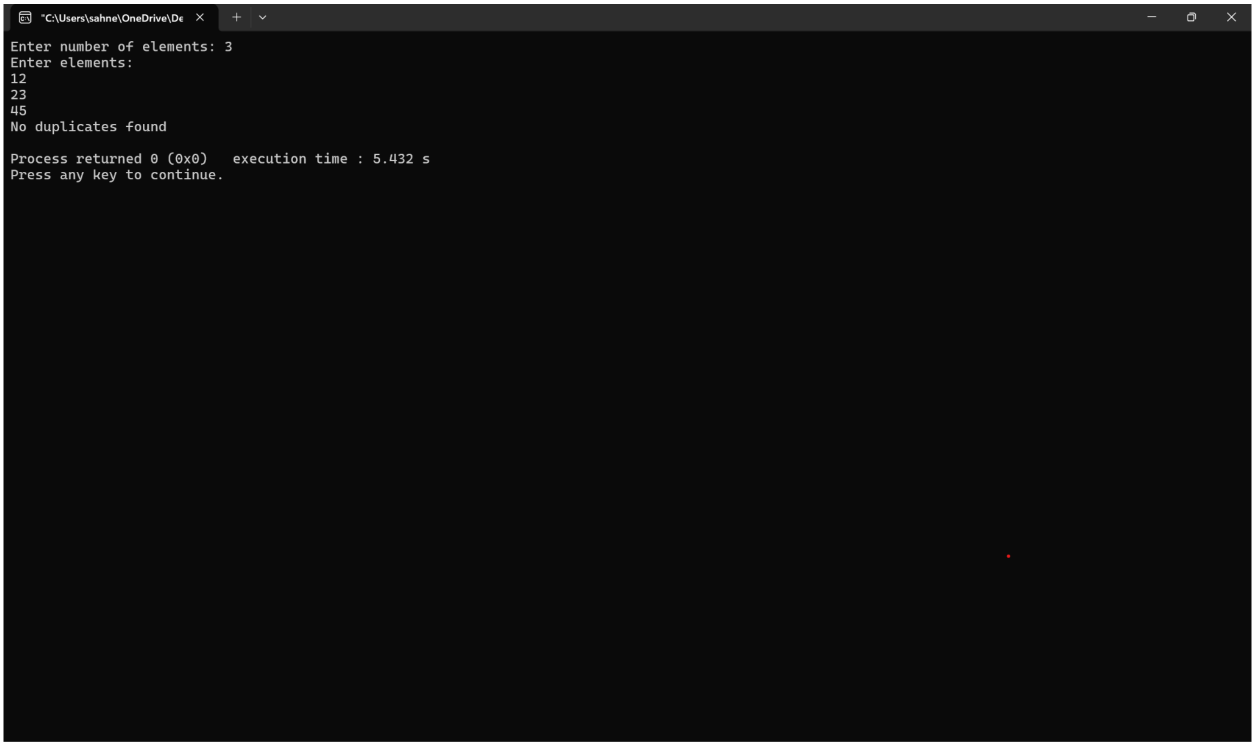
```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int min = arr[0];
    int position = 0;

    for(i = 1; i < n; i++) {
        if(arr[i] < min) {
            min = arr[i];
            position = i;
        }
    }

    printf("Smallest element = %d\n", min);
    printf("Position = %d\n", position + 1);

    return 0;
}
```



3)

```
#include <stdio.h>

int main() {
    int n, i, j, duplicate = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(arr[i] == arr[j]) {
                duplicate = 1;
                printf("Duplicate element found: %d\n", arr[i]);
            }
        }
```

```
    }

    if(duplicate == 0)
        printf("No duplicates found\n");

    return 0;
}
```



Lab program

1)

*fcfs.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Start here *fcfs.c

```c
#include <stdio.h>

int main() {
    int n, i;
    int at[20], bt[20], ct[20], tat[20], wt[20];
    float total_tat = 0, total_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    ct[0] = at[0] + bt[0];

    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }

    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        total_tat += tat[i];
        total_wt += wt[i];
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);

    return 0;
}
```

C:\Users\sahne\OneDrive\Desktop\os lab\fcfs.c C/C++ Windows (CR+LF) WIN

"C:\Users\sahne\OneDrive\De

```
Enter number of processes: 2

Process 1
Arrival Time: 23
Burst Time: 34

Process 2
Arrival Time: 21
Burst Time: 34

P       AT      BT      CT      TAT     WT
1       23      34      57      34      0
2       21      34      91      70      36

Average Turnaround Time = 52.00
Average Waiting Time = 18.00

Process returned 0 (0x0)   execution time : 13.957 s
Press any key to continue.
```